ASK-CARD ORCHESTRATION

Companion explanation for the Costimiser analytical-engine diagram

Architecture note

Core principle: /ask-card interprets and routes a natural-language request. The selected analytical card delegates the work to a specialised tool. That tool contains the main analytical logic, manages access to its dependencies, and returns a result compatible with the common card-response format.

Scope

This document explains the orchestration represented in the accompanying Draw.io diagram. It focuses only on the natural-language entry point **POST /ask-card** and the analytical paths that it can select.

1. Purpose of the orchestration layer

The /ask-card endpoint provides a natural-language interface over the analytical capabilities of the Costimiser engine. It does not duplicate the business logic already implemented in the analytical tools. Instead, it understands the request, normalises the extracted parameters, selects the appropriate card and delegates execution to the corresponding tool.

> ***Only one primary analytical path is selected for a request. The parallel branches in the diagram represent alternative routes, not operations that are executed simultaneously.***

The end-to-end flow is:

1. Natural-language query
2. POST /ask-card
3. Query interpretation
4. Friendly-name resolution
5. Intent and card selection
6. Specialised tool execution
7. Optional enrichment
8. Common response assembly

2. Request interpretation and routing

2.1 Request entry

A user, frontend, notebook or internal service sends a natural-language query to POST /ask-card. The request can also include response options such as `download_artifacts`, `diagnosis_summary` and `cost_driver_summary`.

2.2 Query interpretation

The interpretation stage identifies the analytical intent and extracts the parameters required to execute it. Depending on the request, these parameters may include grade, reel identifier, date range, target period, baseline period, cost or strength function, process variables, optimisation direction, constraints and chart requirements.

2.3 Friendly-name resolution

The natural-language interface distinguishes analytical function names from process-variable names. Functions such as steam, electricity, starch, starch uptake, total, SCT CD and Burst are public analytical names. Process variables used by structured endpoints must match their canonical names, which are available through the process-data variable endpoint.

Inside /ask-card, users may refer to process variables using friendly expressions. The resolution logic translates those expressions into the canonical names required by the selected tool. For example, “current basis weight” can be resolved to `Current_basis_weight`.

2.4 Intent and card router

After interpretation and normalisation, the router selects one primary analytical card. The card represents the request type and provides the bridge between the orchestration layer and the corresponding tool.

3. Responsibilities of cards, tools and dependencies

Component	Primary responsibility	What it does not do
Card	Represents the selected analytical intent, receives normalised parameters, invokes its tool and exposes the result in the common card format.	It does not own the detailed analytical implementation.
Tool	Contains the main analytical logic, validates and prepares inputs, accesses required dependencies, performs the analysis and builds the card result.	It does not require the router to manage data or model access on its behalf.
Dependency	Provides data, models, metadata, storage or external services required by a tool.	It is not an orchestration entry point and is not called directly by the card router.

Architectural rule: Card -> Tool -> Dependencies. The tool owns both the main logic and dependency access.

4. Analytical routes

Process-data card

Process-data tool: Retrieves, filters, compares and visualises process data.

Dependencies managed by the tool: Process-data repository; reel and grade information; canonical variable catalogue.

SHAP card

SHAP tool: Retrieves the relevant data and target model, computes SHAP contributions and builds explanation tables and figures.

Dependencies managed by the tool: Process data; function registry; selected prediction model.

Diagnosis card

Diagnosis tool: Compares a target period with a baseline period and applies the diagnostic hierarchy and summary logic.

Dependencies managed by the tool: Target and baseline data; diagnosis rules; relevant cost and process components.

Cost-drivers card

Cost-drivers tool: Explains the variables or components responsible for a change between target and baseline periods.

Dependencies managed by the tool: Target and baseline data; function resolution; driver-decomposition logic.

Scenario card

Scenario tool: Retrieves or receives a reference snapshot, applies requested interventions, evaluates functions before and after and builds scenario outputs.

Dependencies managed by the tool: Snapshots; canonical variables; function registry; cost and strength models.

Recommendations card

Recommendations tool: Combines analytical evidence with actionable-variable logic and process knowledge to produce operational recommendations.

Dependencies managed by the tool: Cost-driver outputs; process knowledge; model information; optional optimisation results.

Optimisation card

Optimisation tool: Interprets the objective and constraints, retrieves the reference point and bounds, evaluates candidate interventions and returns the best feasible result.

Dependencies managed by the tool: Snapshots; variable bounds; canonical variable catalogue; cost and strength models.

Knowledge / RAG card

Knowledge / RAG tool: Embeds the question, retrieves relevant document chunks and generates a grounded answer.

Dependencies managed by the tool: Papermaking documents; vector database or FAISS; embedding model; language model.

5. Selected result and optional enrichment

All alternative analytical routes converge conceptually into a selected card result. This collector does not combine the outputs of every branch. It represents the result returned by the single card chosen for the current request.

After the primary tool completes, /ask-card may append optional diagnosis or cost-driver summaries when the corresponding request options are enabled. These summaries enrich the primary result; they do not replace the selected analytical operation.

6. Common response assembly

The selected result and any optional enrichment are converted into a common card-response structure. This allows clients to handle all analytical cards consistently.

Response element	Purpose
text	Markdown-formatted explanation or recommendation.
tables	Structured tabular results, returned inline or as downloadable Parquet artifacts.
figures	Plotly figures, returned inline or through downloadable artifact URLs.

6.1 Inline delivery

When download_artifacts is false, tables and figures are embedded directly in the JSON response.

6.2 Artifact delivery

When download_artifacts is true, tables and figures are stored as artifacts and the response contains URLs that the client can retrieve.

7. Example orchestration

For the request:

> *“Minimize steam cost subject to SCT CD >= 2.1 for reel 12602391.”*

1. The query interpreter identifies an optimisation request.
2. The friendly-name resolver recognises steam as the objective function and SCT CD as a constraint function.

3. The router selects the Optimisation card.
4. The card invokes the Optimisation tool with the normalised reel, objective, direction and constraint parameters.
5. The tool retrieves the reference snapshot and variable bounds, accesses the relevant cost and strength models, and executes the optimisation logic.
6. The tool returns the feasible optimum and supporting tables or figures.
7. The common response builder returns the result inline or through artifact URLs.

8. Architectural summary

The design separates natural-language orchestration from specialised analytical implementation. /ask-card is responsible for understanding and routing the request. Cards represent supported analytical intents. Tools contain the main logic and manage all access to data, models, registries and external services. A common response builder standardises the final output for the client.

In one sentence: /ask-card interprets the request, selects one analytical card, delegates execution to a specialised tool that manages its own dependencies, and returns a standardised result containing text, tables and figures.